### M1.1: Strangler Fig Pattern

**Theory:** Places a routing gateway over monolithic apps, proxying paths step-by-step to new microservices until the monolith is retired.

**Details:** Sets route paths in gateway proxies (e.g. Kong). Routes ‘/api/v1/orders‘ to monoliths initially; pivots target endpoints once microservices mature.

**Trade-off:** Requires active anti-corruption adapters and audit processes to hold dual-data streams consistent during shifts.

### M1.2: Anti-Corruption Layer (ACL)

**Theory:** Establishes transformation boundaries between new services and legacy systems to isolate data models.

**Details:** Implements mapping adapters. Clean Domain-Driven entities transform legacy models dynamically, keeping incoming corruptions from leaking.

**Trade-off:** Introduces microsecond translation delays and memory copy footprints, but preserves domain cleanliness.

### M1.3: Dual-Write Database Migration

**Theory:** Performs zero-downtime database migrations via a five-phase writing, auditing, and shifting protocol.

**Details:** Phase 1: Legacy primary. Phase 2: Dual writes (async or sync to new). Phase 3: Run sync reconciliation scripts. Phase 4: Pivot read/writes to new primary. Phase 5: Cut off legacy.

**Trade-off:** Decreases write latency slightly due to dual network writes; requires strict lock matching to prevent drift during syncs.

### M1.4: Outbox Pattern

**Theory:** Binds database state updates to outbox event queues in local transactions, avoiding volatile 2PC protocols.

**Details:** Writes target event payloads into a local ‘outbox‘ table during the primary business transaction. Background readers poll the outbox to publish events.

**Trade-off:** Adds message delivery latency; requires downstream consumers to handle duplicate messages.

### M1.5: Saga Pattern

**Theory:** Coordinates distributed microservice transactions via sequences of local operations and compensating rollback actions.

**Details:** Uses orchestration (central coordinator) or choreography (event-chain triggers). If inventory reserves fail, coordinators execute compensatory refunds.

**Trade-off:** Compensating steps are business-level actions that can fail, requiring manual dead-letter interventions.

### M1.6: Idempotency Keys

**Theory:** Evaluates client request identifiers to prevent duplicate executions from network retries.

**Details:** Sets a Redis unique lock: ‘SET idempotency_key value EX 86400 NX‘. Re-submitted keys bypass execution and return cached outcomes.

**Trade-off:** Adds Redis access overhead; lock expiration times must outlive the longest business transaction lifecycle.

### M2.1: Token & Leaky Bucket Limiting

**Theory:** Filters gateway surges via rate-limiting filters to shield downstream microservices from overload crashes.

**Details:** Token buckets accept transient bursts by storing tokens; leaky buckets enforce fixed egress rates. Handled via Lua scripts in API gateways.

**Trade-off:** Rejects overflow traffic with HTTP 429 errors, degrading client experiences under extreme spikes.

### M2.2: Circuit Breaker Pattern

**Theory:** Cuts off downstream calls dynamically during service failures to prevent thread blockages.

**Details:** Implements CLOSED, OPEN, and HALF_OPEN states. Automatically trips to OPEN when failure rates exceed thresholds, routing calls to fallbacks.

**Trade-off:** Blocks all calls during OPEN states, demanding robust client fallbacks like static mock caches.

### M2.3: Bulkhead Isolation

**Theory:** Segregates core execution thread allocations to isolate service failures.

**Details:** Allocates separate thread pools (e.g. Resilience4j) to different downstream calls. Slow dependencies exhaust their local pool without starving order threads.

**Trade-off:** Elevates context-switching memory costs.

### M2.4: Graceful Degradation

**Theory:** Disables non-critical business logic during extreme load to protect core checkout paths.

**Details:** Switches features (e.g. recommendations) off dynamically via configuration updates, serving static layouts to save CPU cycles.

**Trade-off:** Degrades user feature richness, requiring prioritizations.

### M2.5: Optimistic Locking & ETag

**Theory:** Detects concurrent update clashes using version checks to prevent overwrite losses.

**Details:** Asserts SQL query updates: 'UPDATE tables SET val=x, version=version+1 WHERE id=y AND version=current_version'. Leverages HTTP If-Match for API nodes.

**Trade-off:** Causes high transaction retry rates under heavy concurrent write conflicts.

### M2.6: API Versioning

**Theory:** Maintains backward-compatible interfaces to prevent older client integrations from breaking during updates.

**Details:** Deploys URL namespaces ('/api/v2/'), custom Headers, or Media negotiation types, routing routes through proxies.

**Trade-off:** Increases backend maintenance costs.

### M3.1: Schema Expand-Contract

**Theory:** Modifies database table columns progressively to avoid database locks.

**Details:** Phase 1: Add new column (Expand). Phase 2: Dual-write to both. Phase 3: Backfill data. Phase 4: Shift reads to new column. Phase 5: Drop old column (Contract).

**Trade-off:** Lengthens deployment cycles; demands strict column compatibility checks.

### M3.2: Shadow Database Testing

**Theory:** Benchmarks production nodes under real loads by tagging test traffic to write to isolated databases.

**Details:** Marks test payloads with header tags ('X-Test-Traffic'). Database routers intercept writes and redirect to shadow schemas.

**Trade-off:** Demands synchronized schema updates across both databases; risks production database contamination if routing fails.

### M3.3: Chaos Engineering & Fault Injection

**Theory:** Inject failures (e.g., latency, network drops) proactively to test system resilience.

**Details:** Deploys chaos injection schedules (e.g., Chaos Mesh) to inject delay spikes on microservices, validating fallback routing.

**Trade-off:** Requires strict safety thresholds to prevent system-wide outages.

### M3.4: Data Reconciliation

**Theory:** Validates data parity across nodes or databases, using async audit runs to repair drifts.

**Details:** Compares transaction archives daily. Scans mismatch signatures and runs automated corrections (compensating records).

**Trade-off:** Operates after the fact, introducing temporary inconsistency windows.

### M3.5: Retry Storms & Jitter

**Theory:** Desynchronizes retry spikes during service recovery to prevent overloading downstream servers.

**Details:** Adjusts retry loops with Exponential Backoff and Jitter: 'Interval = Min(Max_Int, Initial_Int * Base^Retry + Jitter)'.

**Trade-off:** Lengthens total request execution delays.

### M3.6: Dead Letter Queue (DLQ)

**Theory:** Isolates corrupt payloads after retry exhaustion to prevent message queue blockages.

**Details:** Routes messages failing processing limits (e.g., 3 retries) to dedicated DLQ queues, alerting engineers.

**Trade-off:** Delays processing; requires monitoring to prevent DLQ build-ups.

### M4.1: Facade Pattern for Monoliths

**Theory:** Encapsulates monolithic interfaces behind unified facade layers to prepare code for microservice splitting.

**Details:** Declares unified service interfaces, blocking raw schema queries.

**Trade-off:** Adds call hops.

### M4.2: Feature Toggles & Canary Launches

**Theory:** Decouples deployment from feature activation, testing new features on a fraction of user traffic.

**Details:** Wraps new features in conditional toggles. Deploys code, then routes 1% of traffic to validate stability.

**Trade-off:** Accumulates code technical debt.

### M4.3: Backpressure Flow Control

**Theory:** Signals upstream producers to slow down when downstream queues overflow.

**Details:** Monitored by downstream queues. Overflowing buffers trigger upstream warnings, returning HTTP 429 to slow clients down.

**Trade-off:** Degrades front-end performance, requiring responsive UI designs.

### M4.4: Zero-Downtime Rolling Updates

**Theory:** Updates application instances incrementally, keeping system capacity stable.

**Details:** Employs Kubernetes deployments, validating new Pods via readiness probes before terminating old instances.

**Trade-off:** Requires extra server resources.

### M4.5: Distributed Lock Leases

**Theory:** Keeps locks active during long transactions by extending lock leases automatically.

**Details:** Redisson-style Watchdog threads ping Redis periodically to extend key expirations.

**Trade-off:** Node failures under long GC pauses risk locking keys; requires fallback transaction timeouts.

### M5.1: CQRS Read/Write Separation

**Theory:** Separates write commands from read queries to optimize performance.

**Details:** Writes to primary database. Fires domain events to sync read models in Elasticsearch.

**Trade-off:** Introduces final consistency delay windows.

### M5.2: <ctrl94>Client-side Load Balancing

**Theory:** Pulls service registry endpoints directly to clients, bypassing gateway bottlenecks.

**Details:** Clients cache service node lists, run round-robin routing locally, and track node error rates.

**Trade-off:** Consumes client-side memory.

### M5.3: Health Check Probes

**Theory:** Monitors container lifecycles via periodic runtime probes, triggering auto-recovers.

**Details:** Configures Liveness (health check restarts) and Readiness (removes broken Pods from load balancer tables) probes.

**Trade-off:** Misconfigured probes can trigger cascading cluster crashes.

### M5.4: Distributed Trace Propagation

**Theory:** Chains request traces across microservice calls to trace system-wide operations.

**Details:** Propagates traceParent headers containing TraceID and SpanID across network boundaries.

**Trade-off:** High collection rates strain storage.

### M5.5: Correlation ID Logs

**Theory:** Associates a unique ID with all logs generated by a request, enabling fast search in ELK.

**Details:** Gateways attach CorrelationIDs to request contexts, printing them in all downstream log nodes.

**Trade-off:** Requires context propagation across thread pools.

### Zone T1: Production Survival Core Libraries

Netflix Resilience4j, Sentinel Core, Kubernetes Probes, Debezium CDC, Spring Cloud Gateway, W3C Trace Context

### Zone T2: System Faults & Diagnostics

cascading_thread_pool_exhaustion_panic, dirty_data_write_skew_migration_error, idempotent_duplicate_submission_intercept, circuit_breaker_open_fast_fail_triggered, poison_pill_max_retry_exhaust_dlq_redirect, watchdog_lease_renewal_lock_expiration